

GPU 재학습 시행착오 기록

ADAS Sound Detector — YAMNet Fine-tuning

작성일: 2026-05-28 ~ 2026-05-30

환경: WSL2 Ubuntu-24.04 | NVIDIA RTX 3070 Laptop | TF 2.21.0

항목	결과
Test Accuracy	96.11%
Best Val Loss	0.0539 (Epoch 51)
Early Stopping	Epoch 61
TFLite 모델 크기	0.67 MB (INT8 양자화)
총 학습 시간	약 1시간 45분

1. 개요

새로 증강된 데이터셋(총 21,425개 → clean 13,885개)으로 YAMNet 파인튜닝 재학습을 시도하는 과정에서 발생한 문제와 해결책을 기록한다. 학습 과정에서 OOM, GPU 오인식, tmux 세션 불안정, Python 환경 미구성 등 다양한 시행착오가 발생하였다.

데이터 구성 (학습 전)

클래스	원본	증강/합성 후	증가 방법
background	2,000	12,000	배경+노이즈 SNR 합성 10,000개 추가
car_horn	429	956	Time stretch / Pitch shift 등 12가지 기법
siren	929	929	변동 없음
합계	3,358	13,885	

2. 시행착오 #1: BOM(Byte Order Mark) 깨짐

발생 상황

PowerShell Out-File 명령으로 run_finetune.sh를 작성할 때 파일 첫 줄에 UTF-8 BOM(0xEF 0xBB 0xBF)이 삽입되어 쉘뱅(#!/bin/bash) 파싱에 실패.

```
/bin/bash: i>¿#!/bin/bash: No such file or directory
```

원인

PowerShell 5.1의 Out-File 기본 인코딩은 UTF-16 LE(BOM 포함)이며, -Encoding utf8 옵션도 UTF-8 with BOM을 출력한다.

영향 및 해결

bash는 첫 줄 오류를 무시하고 나머지를 계속 실행하므로 실제 학습에는 영향 없었음. 근본 해결책: -Encoding ascii 또는 WSL 내에서 직접 cat/echo로 스크립트 작성.

3. 시행착오 #2: OOM(Out of Memory) — 핵심 문제

발생 상황

첫 번째 학습 시도(14:44 시작) 직후, 파형 로딩 완료 후 약 18초 만에 오류 메시지 없이 프로세스가 종료되었다.

```
Loaded 11161 waveforms -- shape (11161, 64000)
```

```
Created device GPU:0 with 5595 MB memory <- GPU ■■■■
Allocation of 2857216000 exceeds 10% of free system memory <- ■■
=== ADAS Finetune End: Thu May 28 14:49:37 KST 2026 === <- ■■ ■■!
```

원인 분석

_dedup_to_clean() 함수가 SNR 파일을 제외했지만, clean 파티션에 synthesize_background.py가 생성한 합성 background 10,000개가 포함되어 훈련 데이터가 11,161개로 부풀었다.

항목	크기 (bytes)	크기 (GB)
X_train (11,161 x 64,000 x float32)	2,857,216,000	2.86 GB
tf.data 복사본 (from_tensor_slices)	2,857,216,000	2.86 GB
X_val + X_test	697,376,000	0.70 GB
Python/TF 오버헤드	—	~0.5 GB
합계	—	약 6.9 GB

WSL2 할당 RAM 7.5 GB 초과 → OS OOM-killer 발동

해결책: background_cap 적용

config.yaml에 background_cap: 2000 추가, finetune.py에 샘플 수 상한 로직 삽입:

```
# config.yaml
finetune:
    background_cap: 2000 # RAM OOM ■■

# finetune.py (fold_split ■■■)
bg_cap = ft_cfg.get('background_cap', 2000)
bg_mask = clean_df['class'] == 'background'
if bg_mask.sum() > bg_cap:
    bg_idx = clean_df[bg_mask].sample(n=bg_cap, random_state=seed).index
    train_df = train_df.loc[bg_idx.union(clean_df[~bg_mask].index)]
```

항목	수정 전	수정 후
train background	9,606개	2,000개
train 전체	11,161개	3,555개
X_train 메모리	2.86 GB	912 MB
결과	OOM 종료	정상 학습 시작

4. 시행착오 #3: 내장 GPU(Radeon) 사용 의심

발생 상황

Windows 작업 관리자에서 WSL2 Python 프로세스가 내장 Radeon GPU 항목 아래에 표시되어 RTX 3070이 사용되지 않는 것으로 오인하였다.

원인 및 해결

Windows 작업 관리자는 WSL2 프로세스의 화면 렌더링을 내장 GPU로 분류하는 특성이 있다. 실제 CUDA 연산은 nvidia-smi로만 정확하게 확인 가능하다.

```
# ■■■ ■■■■  
wsl -d Ubuntu-24.04 nvidia-smi --query-gpu=utilization.gpu,memory.used --format=csv,noheader
```

```
# ■■ ■■ (■■ ■)  
35 %, 7316 MiB    <- RTX 3070 ■■ ■■ ■
```

로그에서 'Created device GPU:0 ... NVIDIA GeForce RTX 3070 Laptop GPU, compute capability: 8.6' 확인 → RTX 3070 정상 사용.

5. 시행착오 #4: tmux 세션 불안정 및 프로세스 종료

발생 상황

PowerShell에서 `wsl -d Ubuntu-24.04 bash -c '...'` 형태로 tmux 세션을 생성하여 학습을 실행했으나, PowerShell 명령이 종료된 후 tmux 세션도 함께 사라지면서 학습 프로세스가 반복적으로 중단되었다. Epoch 34 완료(val_loss=0.0582) 후 저장 전에 종료된 경우도 있었다.

원인

PowerShell에서 wsl 명령을 통해 시작된 백그라운드 프로세스는 PowerShell 세션이 종료될 때 함께 정리된다. `nohup + &` 조합도 WSL 인스턴스 재시작 시 소멸한다.

해결

Windows Terminal에서 Ubuntu 탭을 직접 열어 학습을 포어그라운드로 실행. 터미널 창을 닫지 않으면 프로세스가 안정적으로 유지된다.

```
# Ubuntu ██████ ███ ███
export LD_LIBRARY_PATH=~/.adas-env/lib/python3.12/site-packages/nvidia/...
cd '/mnt/c/Users/Daniel Park/Desktop/SoundPJ-rebuilt'
python3 -m src.finetune --config config.yaml 2>&1 | tee logs/finetune.log
```

6. 시행착오 #5: 새 데스크탑 Python 환경 미구성

발생 상황

학습을 다른 데스크탑에서 이어받으려 했으나 WSL2 가상 환경(venv)이 없어 실행 불가. 추가로 python3.12-venv 패키지 미설치, setuptools 버전 이슈로 pkg_resources 모듈을 찾지 못하는 오류가 연달아 발생.

오류	원인	해결
/home/.../adas-env/bin/python3: No such file or directory	venv가 새 PC에 없음	python3 -m venv adas-env 로 생성
ensurepip is not available	python3.12-venv 미설치	sudo apt install python3.12-venv
ModuleNotFoundError: No module named 'pkg_resources'	setuptools 82.x에서 pkg_resources 분리됨	pip install setuptools==69.5.1

최종 환경 구성 명령어

```
sudo apt update && sudo apt install -y python3.12-venv python3-pip
python3 -m venv ~/.adas-env
source ~/.adas-env/bin/activate
pip install --upgrade pip
pip install tensorflow[and-cuda] librosa soundfile pandas \
```

```
scikit-learn tensorflow-hub pyyaml  
pip install setuptools==69.5.1 # pkg_resources ■■■
```

7. 최종 학습 구성 및 결과

학습 하이퍼파라미터

항목	값
학습 프레임워크	TensorFlow 2.21.0 + CUDA 12.3 + cuDNN 9.2.3
GPU	NVIDIA GeForce RTX 3070 Laptop GPU (8 GB VRAM)
데이터 (train / val / test)	3,555 / 1,360 / 1,364
YAMNet 변수	56개 중 33개(60%) 훈련, 23개 동결
최대 Epochs	100 (Early Stopping patience=10)
Batch size	16
yamnet_lr	5e-6 (YAMNet 미세조정)
clf_lr	2e-5 (분류기 헤드)
background_cap	2,000 (OOM 방지)

주요 Epoch 학습 곡선

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	비고
1	3.411	30.1%	1.015	51.0%	
3	0.767	77.9%	0.215	95.9%	
9	0.247	94.5%	0.135	96.4%	Best (초기)
19	0.107	97.7%	0.130	97.5%	Best 갱신
27	0.065	98.6%	0.121	97.7%	Best 갱신
33	0.070	98.2%	0.094	98.2%	Best 갱신
43	0.035	99.1%	0.080	98.2%	Best 갱신
51	0.027	99.0%	0.054	98.5%	Best 갱신
60	0.015	99.5%	0.057	98.5%	
61	0.027	99.2%	0.157	97.7%	Early Stopping

초록: Best val_loss | 노란: Early Stopping 발동

최종 성능

항목	결과
Early Stopping 발동 Epoch	61
Best Val Loss	0.0539 (Epoch 51)

Test Accuracy	96.11%
학습 소요 시간	약 1시간 45분 (16:39 ~ 18:21 KST)
TFLite INT8 모델	models/adas_detector.tflite (0.67 MB)
Keras 분류기	models/custom_classifier_finetuned.h5 (2.7 MB)

8. 배운 점 요약

1. 메모리 계획의 중요성

대용량 waveform 데이터를 한꺼번에 RAM에 로딩할 때 numpy 배열과 tf.data 복사본의 메모리를 모두 고려해야 한다. 데이터 크기 $\times$ 4 bytes $\times$ 2(복사본) + 모델 메모리 < 가용 RAM 조건 확인 필수.

2. GPU 사용 확인은 nvidia-smi로

Windows 작업 관리자는 WSL2 프로세스를 잘못 분류할 수 있다. 실제 CUDA 사용 여부는 반드시 nvidia-smi로 확인해야 한다.

3. 장시간 학습 프로세스 안정성

PowerShell -> WSL 방식의 백그라운드 실행은 세션 종료 시 프로세스도 종료된다. Ubuntu 터미널을 직접 열어 포어그라운드로 실행하는 것이 가장 안정적이다.

4. Early Stopping의 함정

val_loss가 크게 출렁이는 학습에서는 patience를 넉넉히 주어야 한다. 이번 학습에서 patience 8/10까지 간 뒤 새로운 best를 여러 차례 갱신했다.

5. 환경 이식 시 setuptools 버전

Python 3.12 환경에서 setuptools 70+ 버전은 pkg_resources 모듈이 기본 포함되지 않을 수 있다. setuptools==69.5.1 등 이전 버전으로 다운그레이드하면 해결된다.

본 문서는 ADAS Sound Detector 프로젝트의 GPU 재학습 과정에서 발생한 시행착오를 실시간으로 기록한 보고서입니다.